

Boîte Noire

Vos journaux savent déjà tout, personne ne leur a jamais rien demandé.

Introduction

Vous rejoignez l'équipe technique de Pigeon, une plateforme de messagerie professionnelle qui compte quelques dizaines de milliers d'utilisateurs. Comme toute application sérieuse, Pigeon enregistre ce qui s'y passe : les connexions, les paiements d'abonnement, les erreurs applicatives, les appels à son API publique, les envois de notifications.

Aujourd'hui, ces événements partent dans des fichiers de log que personne ne relit. La direction technique veut les exploiter : qui utilise réellement le produit, quelles erreurs reviennent, quels endpoints sont lents, où les utilisateurs abandonnent.

Votre mission est de construire le service qui **ingère et analyse** ces événements. Rien d'autre. Pas d'interface, pas de gestion d'utilisateurs, pas de reprise de l'existant. Un service Spring Boot, une base MongoDB, et des questions métier auxquelles il faut savoir répondre vite.

Avant d'écrire la moindre ligne de code, votre premier livrable consiste à **rédiger une note de cadrage architectural (ADR)** :

Pourquoi le choix d'une base documentaire s'impose-t-il ici face à l'ajout d'une table SQL ?

Vous devez confronter les deux approches sur ce cas d'usage précis.

Cette justification technique est évaluée **autant que l'implémentation**.



Étape 1 : la note de décision

Avant tout code applicatif, vous produisez une note d'une page qui répond à la question posée en introduction.

Elle doit contenir :

- ce que donnerait une modélisation relationnelle de ces événements, et ses limites concrètes sur ce cas ;
- ce que permet une modélisation documentaire ;
- votre conclusion et le critère qui l'emporte.

Cette note est datée par le commit. Elle doit être poussée sur le dépôt avant le premier commit de code applicatif, et l'ordre est vérifié dans l'historique Git à l'évaluation. Une note écrite après coup ne prouve rien et ne vaut rien.

Indice pour vous mettre sur la piste, sans vous donner la réponse : un événement de connexion, un paiement, une erreur applicative et un appel d'API ne portent pas du tout les mêmes informations.

Étape 2 : le modèle documentaire

Vous concevez la structure des documents et la justifiez.

Exigences :

- **au moins cinq types d'événements** aux structures réellement différentes ;



- **au moins un choix d'embedding et un choix de referencing**, chacun justifié par un critère explicite : taille du document, croissance dans le temps, fréquence de lecture conjointe ;
- des champs communs à tous les événements, identifiés comme tels, qui rendent les recherches transverses possibles.

Rappel : **le code s'écrit en anglais**, donc les noms de collections, de champs et de types d'événements aussi.

Étape 3 : la génération

Une analyse sur trois documents ne prouve rien. Vous devez travailler sur du volume, et ce volume, vous le fabriquez.

Le générateur s'écrit en Java, dans le projet. Une classe dédiée lancée par un profil Spring ou par une commande Maven ou Gradle. Ce n'est pas un script Python, ce n'est pas un script shell : le périmètre de langages du B2 s'applique ici comme partout ailleurs.

Exigences :

- **au moins 100 000 événements** produits ;
- répartis sur **une année simulée** ;
- avec des **volumes réalistes** : des pics en journée, des creux la nuit, des jours plus chargés que d'autres. Une distribution uniforme rend toutes les analyses inintéressantes ;
- avec une population d'utilisateurs cohérente : quelques gros utilisateurs, beaucoup de petits.

Le générateur est un livrable évalué au même titre que le reste. Il doit s'exécuter en une commande, documentée dans le README. :

le code s'écrit en anglais, donc les noms de collections, de champs et de types d'événements aussi.



Étape 4 : les analyses

Vous exposez quatre analyses, chacune sur un endpoint de lecture documenté dans Swagger. Chacune repose sur un **pipeline d'agrégation MongoDB**, pas sur un traitement en mémoire côté Java : charger 100 000 documents pour les compter dans une boucle est exactement ce qu'il faut éviter.

Les quatre analyses imposées :

1. **Top 10 des utilisateurs les plus actifs** sur une période donnée en paramètre.
2. **Répartition des erreurs** par type et par jour, sur une période donnée.
3. **Temps de réponse par endpoint** : moyenne et percentile 95.
4. **Entonnoir de conversion** : combien d'utilisateurs ont enchaîné une séquence d'événements donnée, par exemple inscription puis premier message puis abonnement.

Étape 5 : Optimisation de performance

Vous choisissez la requête la plus coûteuse parmi vos quatre analyses, et vous l'optimisez.

Attendu :

- la mesure explain **avant** index : nombre de documents examinés, nombre de documents retournés, temps ;
- l'index créé, et la justification de son choix, y compris l'ordre des champs s'il est composé ;
- la mesure explain **après** ;



- le constat écrit en trois lignes.

Sur 100 000 documents, l'écart est visible. C'est tout l'intérêt d'avoir généré du volume.

Livrables

1. Note de décision relationnel contre documentaire, 1 page, poussée avant le premier commit de code applicatif.
2. Schéma documentaire commenté, avec justification des choix d'embedding et de referencing.
3. Générateur de données en Java, exécutable en une commande.
4. Quatre analyses exposées en endpoints de lecture, documentées dans Swagger.
5. Dossier de mesure : explain avant, index justifié, explain après, constat.
6. README avec instructions de lancement.

Compétences visées

- **BC02-C3** : concevoir et mettre en place une base de données, niveau Analyser
- **BC02-C4** : développer des composants d'accès aux données SQL et NoSQL
- **BC01-C3** : développer des composants métier



Rendu

Le projet est à rendre sur un dépôt GitHub par binôme :

<https://github.com/prenom-nom/BoiteNoire>

Structure attendue :

BoiteNoire/

src/ (service Spring Boot et générateur)

docs/ (note de décision, schéma documentaire, mesures)

README.md



Base de connaissances

MongoDB

- [Documentation officielle](#)
- [Modélisation des données](#)
- [Embedding contre referencing](#)
- [Pipeline d'agrégation](#)
- [Index](#)
- [Lire un plan d'exécution](#)

Spring

- [Spring Data MongoDB](#)
- [Spring Boot](#)
- [Springdoc OpenAPI](#)

Java

- [Java 21, documentation officielle](#)
- [API Random pour la génération de données](#)